

unordered_map

1. Overview

`unordered_map` is C++'s hash table for storing key-value pairs. It lets you insert, find, update, and erase values by key in average $O(1)$ time.

Example: store each student's roll number and marks.

```
unordered_map<int, int> marks;  
marks[101] = 95;
```

For placements and competitive programming, it is most commonly used for frequency counting, two sum, prefix sums, duplicate detection, graph adjacency by labels, memoization, and fast lookup.

2. Intuition

Think of a hash table like a locker room.

- A key is passed through a hash function.
- The hash function decides which locker/bucket should hold that key.
- Instead of scanning all keys, the table jumps close to where the key should be.
- If multiple keys land in the same bucket, the table handles the collision internally.

Why it works:

1. Good hash functions spread keys across many buckets.
2. Most buckets stay small.
3. Searching inside one small bucket is much faster than searching the whole array.

3. When to Use It

Use `unordered_map` when:

- You need key-value lookup.
- You see trigger phrases like "frequency", "count occurrences", "find if seen before", "first index of value", "last position", "map value to index".
- Keys are not necessarily small or contiguous.
- Average $O(1)$ lookup is enough.
- Order of keys does not matter.
- You need to store custom information for each key.

Common problem signals:

- "Find two numbers with sum K"
- "Count subarrays with sum K"
- "Group by value"

- "Check if a pair exists"
- "Store previous states"
- "Memoize by state"

4. When Not to Use It

Avoid `unordered_map` when:

- You need sorted keys. Use `map` or sorting.
- Keys are small integers from 0 to n . Use a vector for faster and simpler access.
- You need order statistics. Use ordered sets, Fenwick tree, segment tree, or policy-based data structures.
- Worst-case guarantees matter. Hash tables can degrade to $O(n)$ per operation due to collisions.
- You are using pair/vector/custom keys without defining a hash.
- Memory is tight; hash maps use more memory than arrays.

Wrong assumption: average $O(1)$ does not mean guaranteed $O(1)$.

5. Core Concepts

Concept	Meaning	Why It Matters
Key	The lookup identity	Must be hashable and comparable
Value	Data stored against key	Can be count, index, vector, object
Hash function	Converts key to bucket index	Affects performance
Collision	Two keys go to same bucket	Too many collisions slow operations
Load factor	Elements divided by buckets	High load increases collisions
Rehashing	Increasing buckets and redistributing keys	Keeps average operations fast

Small example:

```
unordered_map<string, int> freq;
freq["apple"]++;
freq["banana"]++;
freq["apple"]++;
// apple -> 2, banana -> 1
```

6. Step-by-Step Algorithm

For a typical lookup problem:

1. Create an `unordered_map<Key, Value>`.
2. Iterate through the input.
3. For each element, compute the key you care about.
4. Check if the key already exists using `find`.
5. Use or update the stored value.

6. Insert new keys when needed.
7. Return the final answer.

7. Dry Run

Problem: count frequencies in [4, 2, 4, 3, 2, 4].

Initial state: `freq = {}`.

Step	Element	Operation	Map State
1	4	<code>freq[4]++</code>	<code>{4:1}</code>
2	2	<code>freq[2]++</code>	<code>{4:1, 2:1}</code>
3	4	<code>freq[4]++</code>	<code>{4:2, 2:1}</code>
4	3	<code>freq[3]++</code>	<code>{4:2, 2:1, 3:1}</code>
5	2	<code>freq[2]++</code>	<code>{4:2, 2:2, 3:1}</code>
6	4	<code>freq[4]++</code>	<code>{4:3, 2:2, 3:1}</code>

Final answer: 4 -> 3, 2 -> 2, 3 -> 1.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

unordered_map<int, int> buildFrequencyMap(const vector<int>& nums) {
    unordered_map<int, int> freq;
    freq.reserve(nums.size() * 2);

    for (int x : nums) {
        freq[x]++;
    }
    return freq;
}

int main() {
    vector<int> nums = {4, 2, 4, 3, 2, 4};
    unordered_map<int, int> freq = buildFrequencyMap(nums);

    for (auto [value, count] : freq) {
        cout << value << " appears " << count << " times\n";
    }

    return 0;
}
```

pYTHON IMPLEMENTATION

```

from collections import defaultdict

def build_frequency_map(nums):
    freq = defaultdict(int)
    for x in nums:
        freq[x] += 1
    return dict(freq)

nums = [4, 2, 4, 3, 2, 4]
print(build_frequency_map(nums))

```

10. Code Explanation

- `unordered_map<int, int> freq` stores value to count.
- `reserve(nums.size() * 2)` reduces rehashing for large inputs.
- `freq[x]++` inserts `x` with default `0` if absent, then increments.
- The function returns the completed frequency map.
- In Python, `defaultdict(int)` behaves like C++ `freq[x]++` by starting missing values at `0`.

11. Complexity Analysis

Operation	Average Time	Worst Time	Space
Insert	$O(1)$	$O(n)$	$O(n)$ total
Find	$O(1)$	$O(n)$	$O(1)$ extra
Erase	$O(1)$	$O(n)$	$O(1)$ extra
Build from array	$O(n)$	$O(n^2)$	$O(k)$

`k` is the number of distinct keys.

12. Common Patterns

Pattern	How to Identify	General Approach	Examples
Frequency map	Count occurrences	<code>freq[x]++</code>	Majority Element, Top K Frequent
Value to index	Need previous index	<code>pos[value] = i</code>	Two Sum
Prefix state map	Count earlier prefix states	Store prefix sums	Subarray Sum Equals K
Grouping	Group items by computed key	<code>mp[key].push_back(item)</code>	Group Anagrams
Memoization	Repeated state computation	Cache result by state	DP with string/int state

13. Common Mistakes

- Using `mp[key]` only to check existence; it inserts missing keys.
- Forgetting that iteration order is arbitrary.
- Using `int` for counts or sums that may exceed $2^{31} - 1$.
- Not defining custom hash for pairs.
- Assuming worst-case $O(1)$.
- Forgetting `reserve` in large CP inputs.
- Erasing while iterating incorrectly.

14. Edge Cases

- Empty input.
- Single element.
- All elements same.
- All elements distinct.
- Large negative keys.
- Very large values needing `long long`.
- Input designed to cause collisions.
- Missing key lookup.

15. Variations

Variation	What Changes	Use Case	Importance
<code>unordered_map<K, vector<V>></code>	Multiple values per key	Grouping	High
<code>unordered_map<K, long long></code>	Large counts/sums	OA constraints	High
Custom hash map	User-defined key	Pair/vector/state key	High for CP
<code>map</code>	Sorted keys	Range/order operations	Medium
Vector frequency	Array index as key	Small bounded integers	Very high

16. Related Algorithms/Data Structures

- `unordered_set`: key-only version for existence.
- `map`: ordered key-value pairs with $O(\log n)$.
- Vector/array frequency: faster when keys are small.
- Trie: better for prefix-based string lookup.
- Fenwick/Segment Tree: better for range queries and ordered counts.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Two Sum	LeetCode	Value to index map	Easy

Problem	Platform	Main Idea	Difficulty
First Repeating Element	GFG	Frequency/seen map	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Group Anagrams	LeetCode	Group by sorted/count key	Medium
Subarray Sum Equals K	LeetCode	Prefix sum map	Medium
Top K Frequent Elements	LeetCode	Frequency map + heap/bucket	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Substring	LeetCode	Character count map	Hard
Longest Consecutive Sequence	LeetCode	Hash lookup	Hard
Count Subarrays With Median K	LeetCode	Balance map	Hard

18. Interview Explanation

An `unordered_map` is a hash table storing key-value pairs. I use it when I need fast average lookup, insertion, or frequency counting and the order of keys does not matter. In most interview problems it helps convert a nested search into a single pass by remembering information seen earlier.

19. Revision Notes

- Key idea: store information by key for average $O(1)$ access.
- Use `find` for existence checks without insertion.
- Use `long long` for large counts or prefix sums.
- `reserve` can improve performance.
- Iteration order is not sorted.
- Worst case can degrade under collisions.

20. Final Cheat Sheet

Item	Notes
When to use	Fast key-value lookup, frequency, prefix states
Main operations	<code>mp[key]</code> , <code>mp.find(key)</code> , <code>mp.erase(key)</code>
Average complexity	Insert/find/erase $O(1)$
Key code idea	<code>if (mp.find(key) != mp.end())</code>
Edge cases	Empty input, overflow, missing keys, collisions

unordered_set

1. Overview

`unordered_set` is C++'s hash table for storing unique keys only. It answers "have I seen this value?" in average $O(1)$ time.

It is useful when you do not need a count, index, or attached value.

2. Intuition

Imagine a fast attendance register.

- If a value appears, mark it present.
- Later, ask whether that value has appeared before.
- You do not care how many times it appeared unless the problem asks for count.

The set works because hashing jumps directly to the likely bucket of a key.

3. When to Use It

Use `unordered_set` when:

- You only need existence checks.
- You need to remove duplicates.
- You need to detect repeated values.
- You need membership tests inside loops.
- Problem says "distinct", "unique", "already exists", "seen before", "contains".

4. When Not to Use It

Avoid it when:

- You need counts; use `unordered_map`.
- You need sorted order; use `set`.
- You need duplicates; use `multiset` or map counts.
- Keys are small integers; a boolean vector may be faster.
- You need range queries.

5. Core Concepts

Concept	Meaning	Example
Unique key	Stored once even if inserted multiple times	Insert 5 three times, size still increases once
Membership	Check if key exists	<code>seen.count(x)</code>
Erase	Remove key	<code>seen.erase(x)</code>
Hashing	Maps key to bucket	Internal average fast lookup

6. Step-by-Step Algorithm

For duplicate detection:

1. Create an empty `unordered_set`.
2. Iterate over each element.
3. If the element already exists in the set, duplicate found.
4. Otherwise insert it.
5. If loop ends, no duplicate exists.

7. Dry Run

Input: `[7, 3, 9, 3]`.

Step	Element	Set Before	Action	Result
1	7	<code>{}</code>	insert 7	no duplicate
2	3	<code>{7}</code>	insert 3	no duplicate
3	9	<code>{7,3}</code>	insert 9	no duplicate
4	3	<code>{7,3,9}</code>	already exists	duplicate found

Final answer: `true`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool containsDuplicate(const vector<int>& nums) {
    unordered_set<int> seen;
    seen.reserve(nums.size() * 2);

    for (int x : nums) {
        if (seen.find(x) != seen.end()) {
            return true;
        }
        seen.insert(x);
    }
    return false;
}

int main() {
    vector<int> nums = {7, 3, 9, 3};
    cout << (containsDuplicate(nums) ? "Duplicate found" : "No duplicate") <<
    '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False

print(contains_duplicate([7, 3, 9, 3]))
```

10. Code Explanation

- `seen` stores values already processed.
- `find` checks membership without modifying the set.
- If `x` is found, a duplicate exists immediately.
- Otherwise `x` is inserted for future checks.
- Python's `set` gives the same average $O(1)$ behavior.

11. Complexity Analysis

Operation	Average Time	Worst Time	Space
Insert	$O(1)$	$O(n)$	$O(n)$ total
Lookup	$O(1)$	$O(n)$	$O(1)$ extra
Erase	$O(1)$	$O(n)$	$O(1)$ extra
Duplicate detection	$O(n)$	$O(n^2)$	$O(k)$

12. Common Patterns

Pattern	Identify It	Approach	Examples
Seen before	"appeared earlier"	Insert while scanning	Contains Duplicate
Distinct count	"number of unique values"	Insert all, return size	Distinct Numbers
Fast membership	Repeated <code>exists</code> queries	Preload set	Intersection of Arrays
Sequence starts	Check missing predecessor	Used in longest consecutive	Longest Consecutive Sequence

13. Common Mistakes

- Using `unordered_set` when frequency is required.

- Forgetting duplicate insertion has no effect.
- Expecting sorted output.
- Using `count(x)` repeatedly in very collision-heavy cases without considering worst case.
- Erasing elements needed later.

14. Edge Cases

- Empty array.
- One element.
- All duplicates.
- All unique.
- Negative numbers.
- Large values.
- Strings with different cases, such as "A" and "a".

15. Variations

Variation	What Changes	Use Case	Importance
set	Sorted unique keys	Need order/ranges	Medium
multiset	Allows duplicates	Need sorted duplicate collection	Medium
Boolean vector	Index as key	Small non-negative integers	High
Custom hash set	Custom key type	Pairs/states in CP	High

16. Related Algorithms/Data Structures

- `unordered_map`: use when each key needs a value.
- `set`: use when sorted order matters.
- Bitset: use for dense small integer universe.
- Bloom filter: probabilistic membership, rare in interviews.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Contains Duplicate	LeetCode	Seen set	Easy
Intersection of Two Arrays	LeetCode	Membership set	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Consecutive Sequence	LeetCode	Set starts	Medium/Hard
Happy Number	LeetCode	Detect repeated state	Medium

Problem	Platform	Main Idea	Difficulty
Valid Sudoku	LeetCode	Seen constraints	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Word Ladder	LeetCode	Dictionary set	Hard
Minimum Genetic Mutation	LeetCode	Valid states set	Hard
Similar String Groups	LeetCode	State lookup + DSU/DFS	Hard

18. Interview Explanation

An `unordered_set` is a hash-based container for unique values. I use it when I only need to know whether a value exists. It is especially useful for duplicate detection and membership checks because insert and lookup are average $O(1)$.

19. Revision Notes

- Key idea: unique values with average $O(1)$ membership.
- Use `find(x) != end()` or `count(x)`.
- No ordering.
- No frequency information.
- Use `unordered_map` if counts are needed.

20. Final Cheat Sheet

Item	Notes
When to use	Seen values, uniqueness, membership
Main operations	<code>insert</code> , <code>find</code> , <code>erase</code> , <code>count</code>
Complexity	Average $O(1)$
Key code idea	<code>if (seen.find(x) != seen.end())</code>
Edge cases	Empty input, repeated values, no order

Frequency counting

1. Overview

Frequency counting means counting how many times each value appears. It is one of the most common hashing patterns in interviews and online assessments.

Example: in `[1, 2, 1, 3, 2, 1]`, frequencies are `1 -> 3, 2 -> 2, 3 -> 1`.

2. Intuition

Instead of checking every value against every other value, keep a counter for each value.

Analogy: tally marks on a board.

- See 1: add one tally under 1.
- See 2: add one tally under 2.
- See 1 again: increment 1's tally.

This turns many $O(n^2)$ counting tasks into $O(n)$.

3. When to Use It

Use frequency counting when:

- Problem asks for occurrences.
- You need most/least frequent elements.
- You need compare two arrays/strings by counts.
- You need detect anagrams.
- Problem says "at most K distinct", "exactly K distinct", "frequency", "rearrange", "majority".

4. When Not to Use It

Do not use hash-based counting when:

- Values are small bounded integers and vector count is simpler.
- You need order of occurrence and counts alone are insufficient.
- You need range frequency queries; use prefix counts, Fenwick tree, or wavelet tree.
- You need sorted frequency output; add sorting or use ordered containers.

5. Core Concepts

Concept	Meaning	Example
Frequency map	Value to occurrence count	<code>freq[5] = 3</code>
Distinct count	Number of keys with non-zero count	<code>freq.size()</code>
Character count	Frequency over alphabet	<code>count[26]</code>
Sliding frequency	Maintain counts in current window	Longest substring with K distinct
Count comparison	Compare two frequency structures	Anagram check

6. Step-by-Step Algorithm

1. Create a map or array for counts.
2. Traverse the input.
3. Increment the count for each element.
4. Use the completed counts to answer the problem.
5. If using sliding window, decrement counts when elements leave the window.

6. Remove zero-count keys if distinct count matters.

7. Dry Run

Input string: "aabcabc".

Step	Char	Operation	Frequency
1	a	a++	{a:1}
2	a	a++	{a:2}
3	b	b++	{a:2,b:1}
4	c	c++	{a:2,b:1,c:1}
5	b	b++	{a:2,b:2,c:1}
6	c	c++	{a:2,b:2,c:2}

Final: all three characters have non-zero frequency; a, b, c appear twice.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

unordered_map<char, int> countCharacters(const string& s) {
    unordered_map<char, int> freq;
    for (char ch : s) {
        freq[ch]++;
    }
    return freq;
}

bool areAnagrams(const string& a, const string& b) {
    if (a.size() != b.size()) return false;

    vector<int> count(26, 0);
    for (char ch : a) count[ch - 'a']++;
    for (char ch : b) count[ch - 'a']--;

    for (int x : count) {
        if (x != 0) return false;
    }
    return true;
}

int main() {
    string s = "aabcabc";
    auto freq = countCharacters(s);

    for (auto [ch, count] : freq) {
```

```

        cout << ch << ": " << count << '\n';
    }

    cout << boolalpha << areAnagrams("listen", "silent") << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import Counter

def count_characters(s):
    return Counter(s)

def are_anagrams(a, b):
    return Counter(a) == Counter(b)

print(count_characters("aabcabc"))
print(are_anagrams("listen", "silent"))

```

10. Code Explanation

- `countCharacters` uses `unordered_map<char, int>` for general character counting.
- `freq[ch]++` increments each character count.
- `areAnagrams` uses a fixed array of size 26 because the assumed alphabet is lowercase English letters.
- First string increments counts.
- Second string decrements counts.
- If all counts return to zero, both strings contain exactly the same characters.

11. Complexity Analysis

Task	Time	Space
Count with hash map	$O(n)$ average	$O(k)$
Count lowercase letters	$O(n)$	$O(1)$
Compare two counts	$O(k)$	$O(k)$
Sliding window updates	$O(1)$ average per move	$O(k)$

k is number of distinct values or alphabet size.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Anagram	Same characters, reordered	Compare counts	Valid Anagram

Pattern	How to Identify	Approach	Examples
Majority	More than $n/2$ or $n/3$	Count or Boyer-Moore	Majority Element
Top frequent	Need highest frequency	Count + heap/bucket	Top K Frequent
Distinct in window	Current window counts	Sliding map	Longest Substring with K Distinct
Pair counting	Count complements	Frequency map	Count Pairs with Sum K

13. Common Mistakes

- Not removing keys whose count becomes zero in sliding window.
- Using array count with characters outside expected range.
- Forgetting case sensitivity.
- Using `int` when counts over many combinations need `long long`.
- Sorting when frequency counting would be simpler and faster.
- Comparing maps with extra zero-count keys.

14. Edge Cases

- Empty string or array.
- Single value.
- All same values.
- All distinct values.
- Uppercase/lowercase mix.
- Unicode or non-English characters.
- Negative integers.
- Very large frequency counts.

15. Variations

Variation	What Changes	Use Case	Importance
Fixed array count	Direct index	Small alphabet/range	Very high
Hash map count	Flexible keys	Large values/strings	Very high
Sliding window count	Counts change dynamically	Substring/subarray windows	High
Frequency of frequency	Count how many values have count f	Advanced validation	Medium

16. Related Algorithms/Data Structures

- Sorting: useful when order after grouping matters; $O(n \log n)$.
- Hash map: flexible counting.
- Bucket sort: useful after frequency counting.

- Heap: useful for top K frequent.
- Sliding window: often combined with frequency counts.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Anagram	LeetCode	Character counts	Easy
Majority Element	LeetCode	Count or Boyer-Moore	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Top K Frequent Elements	LeetCode	Count + heap/bucket	Medium
Sort Characters By Frequency	LeetCode	Count + sort	Medium
Longest Substring with At Most K Distinct	LeetCode	Sliding frequency	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Minimum Window Substring	LeetCode	Required frequency	Hard
Subarrays with K Different Integers	LeetCode	Sliding frequency	Hard
Count of Smaller Numbers After Self	LeetCode	Frequency + Fenwick/compression	Hard

18. Interview Explanation

Frequency counting stores how many times each value appears. I use a hash map for general values and a fixed array for small alphabets or bounded integers. It is useful because it turns repeated searching into direct count updates and lookups.

19. Revision Notes

- Key idea: value -> count.
- Use vector count for small ranges.
- Use `unordered_map` for large or unknown keys.
- Remove zero-count keys in sliding windows.
- Counting is usually $O(n)$.

20. Final Cheat Sheet

Item	Notes
When to use	Occurrences, anagrams, top K, distinct windows

Item	Notes
Main operations	Increment, decrement, compare, erase zero
Complexity	$O(n)$ average
Key code idea	<code>freq[x]++</code>
Edge cases	Empty input, zero counts, case sensitivity, overflow

Pair sum / two sum

1. Overview

Pair sum or two sum asks whether two elements combine to a target, usually by addition. The classic problem returns indices `i` and `j` such that `nums[i] + nums[j] == target`.

Hashing solves it in one pass by storing previously seen values.

2. Intuition

For every number `x`, the required partner is `target - x`.

Instead of searching the whole remaining array for that partner, remember values already seen.

Step reasoning:

1. At current value `x`, compute `need = target - x`.
2. If `need` was seen before, answer found.
3. Otherwise store `x` for future elements.

This works because every valid pair has one element that appears earlier in the scan and one that appears later.

3. When to Use It

Use this pattern when:

- Problem asks for two values adding to target.
- You need indices of a pair.
- Problem says "pair exists", "find pair", "sum equals K", "complement".
- Array is unsorted and you want $O(n)$.
- You need count of pairs with a given sum.

4. When Not to Use It

Avoid hash two sum when:

- Array is sorted and two pointers are simpler with $O(1)$ extra space.
- You need all unique pairs in sorted order; sorting plus two pointers is cleaner.
- You need closest pair sum; sorting/two pointers is better.

- Input values are tiny and frequency vector is simpler.
- You need pair under range/order constraints not captured by hash lookup.

5. Core Concepts

Concept	Meaning	Example
Complement	Needed partner	<code>target - x</code>
Seen map	Stores previous values and indices	<code>index[value] = i</code>
One-pass	Check before insert	Avoid using same element twice
Duplicate handling	Same value may be needed twice	Target 6, pair 3 + 3
Pair count	Count earlier complements	Add <code>freq[need]</code>

6. Step-by-Step Algorithm

For returning one pair of indices:

1. Create `unordered_map<int, int> indexByValue`.
2. Traverse the array from left to right.
3. For current `nums[i]`, compute `need = target - nums[i]`.
4. If `need` exists in map, return `{indexByValue[need], i}`.
5. Otherwise store `nums[i] -> i`.
6. If no pair is found, return empty result.

7. Dry Run

Input: `nums = [2, 7, 11, 15]`, `target = 9`.

i	nums[i]	need	Map Before	Action
0	2	7	<code>{}</code>	store 2 -> 0
1	7	2	<code>{2:0}</code>	found need 2

Final answer: indices `[0, 1]`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> twoSum(const vector<int>& nums, int target) {
    unordered_map<int, int> indexByValue;
    indexByValue.reserve(nums.size() * 2);

    for (int i = 0; i < (int)nums.size(); i++) {
        int need = target - nums[i];
```

```

        auto it = indexByValue.find(need);
        if (it != indexByValue.end()) {
            return {it->second, i};
        }

        indexByValue[nums[i]] = i;
    }

    return {};
}

long long countPairsWithSumK(const vector<int>& nums, int k) {
    unordered_map<int, long long> freq;
    long long pairs = 0;

    for (int x : nums) {
        int need = k - x;
        if (freq.find(need) != freq.end()) {
            pairs += freq[need];
        }
        freq[x]++;
    }

    return pairs;
}

int main() {
    vector<int> nums = {2, 7, 11, 15};
    int target = 9;

    vector<int> answer = twoSum(nums, target);
    if (!answer.empty()) {
        cout << answer[0] << " " << answer[1] << '\n';
    }

    cout << countPairsWithSumK({1, 5, 7, -1, 5}, 6) << '\n';
    return 0;
}

```

PYTHON IMPLEMENTATION

```

from collections import defaultdict

def two_sum(nums, target):
    index_by_value = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in index_by_value:
            return [index_by_value[need], i]
        index_by_value[x] = i
    return []

```

```
def count_pairs_with_sum_k(nums, k):
    freq = defaultdict(int)
    pairs = 0
    for x in nums:
        pairs += freq[k - x]
        freq[x] += 1
    return pairs

print(two_sum([2, 7, 11, 15], 9))
print(count_pairs_with_sum_k([1, 5, 7, -1, 5], 6))
```

10. Code Explanation

- `indexByValue` stores values already processed and their indices.
- `need = target - nums[i]` is the only possible partner for current value.
- Checking before insertion prevents using the same index twice.
- `countPairsWithSumK` adds the number of earlier complements, then records the current value.
- `long long` is used for pair count because the number of pairs can be $O(n^2)$.

11. Complexity Analysis

Variant	Time	Space	Notes
One pair with hash map	$O(n)$ average	$O(n)$	Best for unsorted array
Count pairs	$O(n)$ average	$O(k)$	Handles duplicates
Sorted two pointers	$O(n \log n)$ sort + $O(n)$	$O(1)$ or $O(n)$	Good for all pairs
Brute force	$O(n^2)$	$O(1)$	Only for tiny input

12. Common Patterns

Pattern	How to Identify	General Approach	Examples
Return indices	"return indices"	Map value to index	Two Sum
Count pairs	"number of pairs"	Add <code>freq[target - x]</code>	Count Pairs with Given Sum
Unique pairs	"unique pairs"	Sort + skip duplicates	3Sum base idea
Pair with difference K	<code>abs(a-b)=k</code>	Store/search <code>x-k</code> , <code>x+k</code>	K-diff Pairs

13. Common Mistakes

- Inserting current element before checking complement and using the same element twice.
- Overwriting an earlier index when the problem requires first pair.
- Not handling duplicates like `[3, 3]`, target 6.
- Using `int` for pair count.
- Returning values when the problem asks for indices.

- Forgetting negative numbers are valid.

14. Edge Cases

- No pair exists.
- Exactly two elements.
- Same value needed twice.
- Multiple valid answers.
- Negative numbers.
- Zero target.
- Large values causing overflow in `target - x`; use `long long` if constraints require.

15. Variations

Variation	What Changes	Use Case	Importance
Sorted two sum	Use two pointers	Sorted input or unique pairs	High
Count pair sum	Use frequency counts	Count all valid pairs	High
3Sum/4Sum	Fix elements + two sum	Multi-element sum	Very high
Two sum in BST	Inorder/two-set	Tree input	Medium
Pair difference	Complement becomes <code>x-k/x+k</code>	Difference problems	High

16. Related Algorithms/Data Structures

- Two pointers: better when array is sorted.
- Sorting: useful for unique pairs and duplicate control.
- Frequency map: count pair multiplicities.
- Prefix sum + hash map: subarray equivalent of two sum.
- Meet-in-the-middle: larger subset sum variants.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Two Sum	LeetCode	Complement lookup	Easy
Pair with Given Sum	GFG	Frequency/complement	Easy

Medium

Problem	Platform	Main Idea	Difficulty
3Sum	LeetCode	Sort + two pointers	Medium
K-diff Pairs in an Array	LeetCode	Frequency/set	Medium

Problem	Platform	Main Idea	Difficulty
Count Good Meals	LeetCode	Count complements over powers of two	Medium

Hard

Problem	Platform	Main Idea	Difficulty
4Sum	LeetCode	Sort + two pointers	Hard-ish
Max Number of K-Sum Pairs	LeetCode	Count/remove pairs	Medium/Hard OA
Count Pairs With XOR in Range	LeetCode	Trie instead of sum hash	Hard

18. Interview Explanation

For two sum, I scan once and store previously seen values in a hash map. For each value, I compute the complement needed to reach the target. If the complement is already in the map, I have found the pair; otherwise I store the current value.

19. Revision Notes

- Key formula: $need = target - x$.
- Check before insert.
- Map value to index for returning indices.
- Map value to frequency for counting pairs.
- Use two pointers if sorted.

20. Final Cheat Sheet

Item	Notes
When to use	Pair sum in unsorted array
Main operations	Complement lookup, insert current
Complexity	$O(n)$ average, $O(n)$ space
Key code idea	<code>if (mp.count(target - x))</code>
Edge cases	Duplicates, no pair, same value twice, overflow

Subarray sum with hashmap

1. Overview

Subarray sum with hashmap is a prefix-sum technique used to find or count contiguous subarrays whose sum equals a target.

The classic problem is: count subarrays with sum k .

2. Intuition

Let `prefix[i]` be the sum from the start to index `i`.

Sum of subarray `l..r` is:

$$\text{prefix}[r] - \text{prefix}[l - 1]$$

If we want this sum to be `k`, then:

$$\text{prefix}[l - 1] = \text{prefix}[r] - k$$

So while scanning, for current prefix sum `prefix`, count how many earlier prefix sums equal `prefix - k`.

3. When to Use It

Use it when:

- Problem asks for contiguous subarray sum.
- Array has negative numbers, so sliding window may fail.
- Problem says "count subarrays with sum K".
- Problem involves equal number of two categories after converting values.
- Need longest subarray with given sum.
- Need subarray divisible by `k`.

4. When Not to Use It

Avoid it when:

- All numbers are positive and you only need existence or shortest/longest; sliding window may be simpler.
- Problem asks subsequence, not subarray.
- You need range updates or many online queries; use Fenwick/segment tree variants.
- Values are huge and sums may overflow if using `int`.

5. Core Concepts

Concept	Meaning	Why It Matters
Prefix sum	Sum from index <code>0</code> to current	Represents state before/after subarray
Earlier prefix	Prefix before subarray starts	Needed to form target sum
Frequency map	Count of prefix sums seen	Counts all possible starts
Initial <code>freq[0]=1</code>	Empty prefix before array	Handles subarrays starting at index <code>0</code>
Remainder map	Store prefix modulo <code>k</code>	Divisibility variants

6. Step-by-Step Algorithm

For counting subarrays with sum k :

1. Set `prefix = 0`.
2. Create map `freq` and set `freq[0] = 1`.
3. Initialize `answer = 0`.
4. For each element x :
5. Add x to `prefix`.
6. Add `freq[prefix - k]` to `answer`.
7. Increment `freq[prefix]`.
8. Return `answer`.

7. Dry Run

Input: `nums = [1, 2, 3, -2, 2]`, $k = 3$.

Initial: `prefix = 0`, `freq = {0:1}`, `answer = 0`.

i	x	prefix	need prefix-k	freq[need]	answer	freq update
0	1	1	-2	0	0	<code>freq[1]=1</code>
1	2	3	0	1	1	<code>freq[3]=1</code>
2	3	6	3	1	2	<code>freq[6]=1</code>
3	-2	4	1	1	3	<code>freq[4]=1</code>
4	2	6	3	1	4	<code>freq[6]=2</code>

Final answer: `4`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long countSubarraysWithSumK(const vector<int>& nums, long long k) {
    unordered_map<long long, long long> prefixFreq;
    prefixFreq.reserve(nums.size() * 2);

    long long prefix = 0;
    long long answer = 0;
    prefixFreq[0] = 1;

    for (int x : nums) {
        prefix += x;
        long long need = prefix - k;

        if (prefixFreq.find(need) != prefixFreq.end()) {
            answer += prefixFreq[need];
        }
    }

    return answer;
}
```

```

    }

    prefixFreq[prefix]++;
}

return answer;
}

int longestSubarrayWithSumK(const vector<int>& nums, long long k) {
    unordered_map<long long, int> firstIndex;
    long long prefix = 0;
    int best = 0;
    firstIndex[0] = -1;

    for (int i = 0; i < (int)nums.size(); i++) {
        prefix += nums[i];

        if (firstIndex.find(prefix - k) != firstIndex.end()) {
            best = max(best, i - firstIndex[prefix - k]);
        }

        if (firstIndex.find(prefix) == firstIndex.end()) {
            firstIndex[prefix] = i;
        }
    }

    return best;
}

int main() {
    vector<int> nums = {1, 2, 3, -2, 2};
    cout << countSubarraysWithSumK(nums, 3) << '\n';
    cout << longestSubarrayWithSumK(nums, 3) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import defaultdict

def count_subarrays_with_sum_k(nums, k):
    freq = defaultdict(int)
    freq[0] = 1
    prefix = 0
    answer = 0

    for x in nums:
        prefix += x
        answer += freq[prefix - k]
        freq[prefix] += 1

```

```

    return answer

def longest_subarray_with_sum_k(nums, k):
    first_index = {0: -1}
    prefix = 0
    best = 0

    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first_index:
            best = max(best, i - first_index[prefix - k])
        if prefix not in first_index:
            first_index[prefix] = i

    return best

print(count_subarrays_with_sum_k([1, 2, 3, -2, 2], 3))
print(longest_subarray_with_sum_k([1, 2, 3, -2, 2], 3))

```

10. Code Explanation

- `prefixFreq[0] = 1` represents the empty prefix before the array starts.
- `prefix += x` updates the sum ending at current index.
- `need = prefix - k` is the prefix sum that must have appeared before.
- `answer += prefixFreq[need]` counts all valid starting positions.
- For longest length, we store the first index of each prefix sum, because the earliest start gives maximum length.
- `long long` prevents overflow when many values are added.

11. Complexity Analysis

Operation	Time	Space
Build prefix while scanning	$O(n)$	$O(1)$ extra besides map
Hash lookup/update	$O(1)$ average	$O(n)$ map
Count subarrays	$O(n)$ average	$O(n)$
Longest subarray	$O(n)$ average	$O(n)$

Worst-case hash collision time can degrade to $O(n^2)$.

12. Common Patterns

Pattern	How to Identify	General Approach	Examples
Sum equals K	Count contiguous subarrays	Prefix frequency	Subarray Sum Equals K
Longest sum K	Need max length	Prefix first index	Longest Subarray Sum K
Divisible by K	Sum % K equals 0	Remainder frequency	Subarray Sums Divisible by K

Pattern	How to Identify	General Approach	Examples
Equal 0 and 1	Convert 0 to -1	Prefix sum 0	Contiguous Array
Binary goal	Count sum in binary array	Prefix or sliding	Binary Subarrays With Sum

13. Common Mistakes

- Forgetting `freq[0] = 1`.
- Using sliding window when negative numbers exist.
- Updating `freq[prefix]` before counting, which can count invalid empty subarrays for some cases.
- Using `int` for prefix sum.
- For longest length, overwriting the first index.
- Mishandling negative modulo in divisibility variants.

14. Edge Cases

- Empty array.
- Single element equal to `k`.
- Subarray starts at index `0`.
- Negative numbers.
- Zeros.
- `k = 0`.
- Large values.
- Multiple identical prefix sums.

15. Variations

Variation	What Changes	Use Case	Importance
Count sum K	Store prefix frequencies	Number of subarrays	Very high
Longest sum K	Store first prefix index	Maximum length	High
Divisible by K	Store prefix modulo	Remainder matching	High
2D submatrix sum	Compress rows + prefix hash	Matrix problems	Medium/Hard
XOR subarray	Prefix XOR instead of sum	XOR target	High

16. Related Algorithms/Data Structures

- Sliding window: works better for non-negative arrays.
- Two sum: prefix-sum hashmap is the subarray version of complement lookup.
- Fenwick tree: useful for prefix sums with updates.
- Segment tree: useful for range queries/updates.
- Prefix XOR: same idea using XOR operation.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Largest Subarray with 0 Sum	GFG	Prefix first index	Easy/Medium
Contiguous Array	LeetCode	Convert 0 to -1	Easy/Medium

Medium

Problem	Platform	Main Idea	Difficulty
Subarray Sum Equals K	LeetCode	Prefix frequency	Medium
Subarray Sums Divisible by K	LeetCode	Remainder frequency	Medium
Binary Subarrays With Sum	LeetCode	Prefix frequency	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Count Number of Nice Subarrays	LeetCode	Prefix count of odds	Medium/Hard
Number of Submatrices That Sum to Target	LeetCode	2D compression + hashmap	Hard
Count Subarrays With Fixed Bounds	LeetCode	Boundary tracking variant	Hard

18. Interview Explanation

I use prefix sums to represent every subarray sum as a difference between two prefix sums. While scanning, if the current prefix is p , then a previous prefix $p-k$ would form a subarray ending here with sum k . A hashmap stores how many times each previous prefix has occurred.

19. Revision Notes

- Formula: $\text{subarray}(l, r) = \text{prefix}[r] - \text{prefix}[l-1]$.
- Need previous prefix: $\text{prefix} - k$.
- Initialize $\text{freq}[0] = 1$.
- Use `long long`.
- Negative numbers are allowed; sliding window is not reliable there.

20. Final Cheat Sheet

Item	Notes
When to use	Contiguous subarray sum with negatives
Main operations	Prefix update, lookup $\text{prefix}-k$, increment prefix count
Complexity	$O(n)$ average
Key code idea	<code>ans += freq[prefix - k]</code>
Edge cases	Starts at 0, $k=0$, negatives, overflow

Longest consecutive sequence

1. Overview

Longest consecutive sequence asks for the length of the longest run of integers that appear consecutively, regardless of their positions in the input.

Example: `[100, 4, 200, 1, 3, 2]` has sequence `1,2,3,4`, so answer is `4`.

2. Intuition

A number can start a consecutive sequence only if $x - 1$ is not present.

So:

1. Put all numbers in a hash set.
2. For each number x , only start counting if $x - 1$ is missing.
3. Count x , $x+1$, $x+2 \dots$ while present.

This avoids recounting the same sequence from every number.

3. When to Use It

Use it when:

- Need longest consecutive values, not necessarily adjacent indices.
- Input is unsorted.
- Need better than $O(n \log n)$ sorting.
- Problem says "consecutive elements", "longest streak", "sequence length".

4. When Not to Use It

Avoid hash-set method when:

- The array is already sorted and duplicates can be skipped directly.
- You need actual sorted sequence output with order; sorting may be clearer.
- Values are in a tiny range; boolean array may be faster.
- You need longest increasing subsequence by position; that is LIS, not this problem.

5. Core Concepts

Concept	Meaning	Why It Matters
Hash set	Stores all unique values	Fast membership
Sequence start	$x-1$ missing	Prevents repeated counting
Forward expansion	Check $x+1$, $x+2$	Measures streak
Duplicates	Ignored by set	Do not inflate length
Value order	Consecutive by value, not index	Different from subarray/subsequence

6. Step-by-Step Algorithm

1. Insert all numbers into an `unordered_set`.
2. Initialize `best = 0`.
3. For each number `x` in the set:
4. If `x - 1` exists, skip `x`.
5. Otherwise, `x` is a sequence start.
6. Keep incrementing `current` while it exists in the set.
7. Update `best` with the sequence length.
8. Return `best`.

7. Dry Run

Input: `[100, 4, 200, 1, 3, 2]`.

Set: `{100, 4, 200, 1, 3, 2}`.

x	Is x-1 present?	Action	Length
100	no 99	start 100	1
4	yes 3	skip	-
200	no 199	start 200	1
1	no 0	start 1,2,3,4	4
3	yes 2	skip	-
2	yes 1	skip	-

Final answer: 4.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int longestConsecutive(const vector<int>& nums) {
    unordered_set<int> values(nums.begin(), nums.end());
    int best = 0;

    for (int x : values) {
        if (values.find(x - 1) != values.end()) {
            continue;
        }

        int current = x;
        int length = 1;

        while (values.find(current + 1) != values.end()) {
            current++;
        }
    }
}
```

```

        length++;
    }

    best = max(best, length);
}

return best;
}

int main() {
    vector<int> nums = {100, 4, 200, 1, 3, 2};
    cout << longestConsecutive(nums) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def longest_consecutive(nums):
    values = set(nums)
    best = 0

    for x in values:
        if x - 1 in values:
            continue

        current = x
        length = 1
        while current + 1 in values:
            current += 1
            length += 1

        best = max(best, length)

    return best

print(longest_consecutive([100, 4, 200, 1, 3, 2]))

```

10. Code Explanation

- The set removes duplicates and provides average $O(1)$ membership.
- $x - 1$ check identifies sequence starts.
- The while loop expands only from starts.
- Each number is part of at most one forward expansion from the start of its sequence.
- `best` stores the maximum length found.

11. Complexity Analysis

Step	Time	Space
Build set	$O(n)$ average	$O(n)$
Iterate starts	$O(n)$ average	$O(1)$ extra
Forward expansions	$O(n)$ total average	$O(1)$ extra
Overall	$O(n)$ average	$O(n)$

Worst-case hash collisions can degrade time.

12. Common Patterns

Pattern	How to Identify	General Approach	Examples
Consecutive streak	Values form runs	Hash set starts	Longest Consecutive Sequence
Missing predecessor	Start condition	Only expand from starts	Sequence problems
Deduplicate first	Duplicates irrelevant	Use set	Distinct value runs
Small range	Bounded values	Boolean array	Presence marking

13. Common Mistakes

- Counting from every number, causing $O(n^2)$.
- Sorting and forgetting to skip duplicates.
- Confusing consecutive sequence with increasing subsequence.
- Overflow when checking $x - 1$ or $x + 1$ near integer limits.
- Iterating original array and doing duplicate work; iterate set if possible.

14. Edge Cases

- Empty array.
- One element.
- All duplicates.
- Already consecutive.
- Negative values.
- Large gaps.
- Values near `INT_MIN` or `INT_MAX`.

15. Variations

Variation	What Changes	Use Case	Importance
Sorting approach	Sort and count runs	Simpler, $O(n \log n)$	High
DSU approach	Union neighboring values	Dynamic/extended versions	Medium
Boolean presence	Direct index	Small value range	Medium
Return actual sequence	Store start and length	Need output elements	Medium

16. Related Algorithms/Data Structures

- `unordered_set`: core lookup structure.
- Sorting: alternative with simpler worst-case reasoning.
- LIS: for index-ordered increasing subsequence, not same.
- DSU: connects neighboring values in graph-like variations.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Missing Number	LeetCode	Presence/sum	Easy
Contains Duplicate	LeetCode	Set membership	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Longest Consecutive Sequence	LeetCode	Hash set starts	Medium/Hard
Array Nesting	LeetCode	Visited chains	Medium
Find All Numbers Disappeared in an Array	LeetCode	Presence marking	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Making A Large Island	LeetCode	Component IDs + set	Hard
Longest Duplicate Subarray variants	Codeforces	Hashing/sets	Hard
Dynamic Connectivity of Consecutive Values	Custom/CP	DSU + hash map	Hard

18. Interview Explanation

I put all values in a hash set. A value starts a consecutive run only when its predecessor is absent. From such starts, I count forward while the next value exists. This ensures every run is counted once and gives average linear time.

19. Revision Notes

- Start only if $x - 1$ missing.
- Expand with $x + 1$.
- Duplicates are removed by set.
- Average $O(n)$, space $O(n)$.
- Watch integer overflow at boundaries.

20. Final Cheat Sheet

Item	Notes
When to use	Longest consecutive values, unsorted input
Main operations	Build set, check predecessor, expand
Complexity	$O(n)$ average
Key code idea	<code>if (!set.count(x-1)) start</code>
Edge cases	Empty, duplicates, negative values, int limits

Detect duplicates

1. Overview

Duplicate detection checks whether the same value appears more than once. Hashing solves this by keeping a set of values seen so far.

There are also variants: find the duplicate value, count duplicates, return duplicate indices, or detect duplicates within distance k .

2. Intuition

As you scan the array, every value is either new or already seen.

- New: store it.
- Already seen: duplicate found.

This is like checking IDs at an entry gate. If an ID has already entered, the second entry reveals a duplicate.

3. When to Use It

Use hashing when:

- Need check if any duplicate exists.
- Need find repeated values in unsorted data.
- Need detect duplicates within a sliding window.
- Problem says "appears twice", "repeated", "contains duplicate", "seen before".
- Values are large or negative.

4. When Not to Use It

Avoid hash set when:

- Array can be sorted and extra space must be $O(1)$.
- Values are from $1..n$ and in-place marking or cycle detection is required.
- You need counts, not just existence; use map.
- You need sorted duplicate output.
- Memory is very limited.

5. Core Concepts

Concept	Meaning	Example
Seen set	Values already visited	{1, 4, 9}
Duplicate	A value that appears again	second 4
Window duplicate	Duplicate within distance k	indices differ by at most k
Count duplicate	Need frequency > 1	use map
In-place duplicate	Modify array to mark visits	only for constrained values

6. Step-by-Step Algorithm

For any duplicate:

1. Create empty hash set `seen`.
2. For each value x :
3. If x exists in `seen`, return `true`.
4. Insert x .
5. Return `false` after the loop.

For duplicates within distance k :

1. Maintain a set of last k elements.
2. Before inserting `nums[i]`, check if it exists.
3. Insert `nums[i]`.
4. If window size exceeds k , remove `nums[i-k]`.

7. Dry Run

Input: [1, 2, 3, 1].

i	x	Seen Before	Action
0	1	{}	insert 1
1	2	{1}	insert 2
2	3	{1,2}	insert 3
3	1	{1,2,3}	duplicate found

Final answer: `true`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool containsDuplicate(const vector<int>& nums) {
```

```

unordered_set<int> seen;
seen.reserve(nums.size() * 2);

for (int x : nums) {
    if (seen.find(x) != seen.end()) {
        return true;
    }
    seen.insert(x);
}
return false;
}

bool containsNearbyDuplicate(const vector<int>& nums, int k) {
    unordered_set<int> window;

    for (int i = 0; i < (int)nums.size(); i++) {
        if (window.find(nums[i]) != window.end()) {
            return true;
        }

        window.insert(nums[i]);

        if ((int>window.size() > k) {
            window.erase(nums[i - k]);
        }
    }

    return false;
}

int main() {
    cout << boolalpha;
    cout << containsDuplicate({1, 2, 3, 1}) << '\n';
    cout << containsNearbyDuplicate({1, 2, 3, 1}, 3) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False

def contains_nearby_duplicate(nums, k):
    window = set()
    for i, x in enumerate(nums):
        if x in window:

```

```
        return True
    window.add(x)
    if len(window) > k:
        window.remove(nums[i - k])
    return False

print(contains_duplicate([1, 2, 3, 1]))
print(contains_nearby_duplicate([1, 2, 3, 1], 3))
```

10. Code Explanation

- `containsDuplicate` stores every seen number.
- If the current number is already present, there is a duplicate.
- `containsNearbyDuplicate` keeps only the last `k` elements in the set.
- Removing `nums[i-k]` ensures the window represents valid index distance.
- The set handles negative and large values naturally.

11. Complexity Analysis

Variant	Time	Space
Any duplicate	$O(n)$ average	$O(n)$
Nearby duplicate	$O(n)$ average	$O(k)$
Sorting approach	$O(n \log n)$	$O(1)$ or $O(n)$
In-place marking	$O(n)$	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Any duplicate	"contains duplicate"	Seen set	Contains Duplicate
Nearby duplicate	index distance condition	Sliding set	Contains Duplicate II
Almost duplicate	value and index distance	Balanced BST/bucket	Contains Duplicate III
Duplicate number	values $1..n$	Cycle detection	Find Duplicate Number

13. Common Mistakes

- Using set when the problem asks for duplicate count.
- Forgetting to remove old elements in distance-based duplicate problems.
- Removing the wrong index from the sliding window.
- Assuming sorted duplicate detection preserves original indices.
- Using in-place marking when values can be negative or outside range.

14. Edge Cases

- Empty array.
- Single element.
- All same values.
- No duplicates.
- Duplicate at start and end.
- $k = 0$ for nearby duplicate.
- Negative values.
- Large values.

15. Variations

Variation	What Changes	Use Case	Importance
Any duplicate	Set only	Basic detection	Very high
Count duplicates	Map counts	Need frequencies	High
Nearby duplicate	Sliding set	Distance constraint	High
Almost duplicate	Ordered set or buckets	Value distance too	Medium/Hard
Find duplicate number	Floyd cycle	$1..n$, no modification	High

16. Related Algorithms/Data Structures

- `unordered_set`: fastest average duplicate detection.
- Sorting: detects adjacent equal values after sort.
- Frequency map: when counts matter.
- Sliding window: for index-distance variants.
- Floyd cycle detection: for special duplicate-number constraints.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Contains Duplicate	LeetCode	Seen set	Easy
Find the Duplicate Number in Array	GFG	Frequency or marking	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Contains Duplicate II	LeetCode	Sliding set	Medium
Find the Duplicate Number	LeetCode	Floyd cycle	Medium
Repeated DNA Sequences	LeetCode	Hash repeated strings	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Contains Duplicate III	LeetCode	Buckets or ordered set	Hard
First Missing Positive	LeetCode	In-place marking	Hard
Duplicate Subtrees	LeetCode	Hash serialized states	Hard

18. Interview Explanation

To detect duplicates, I scan the array and maintain a hash set of values already seen. If a value appears again, it must be a duplicate. This gives average linear time and works for unsorted arrays with large or negative values.

19. Revision Notes

- Use set for existence.
- Use map for counts.
- Nearby duplicate needs sliding set of size k .
- Sorting is valid if modifying order is allowed.
- Special $1..n$ duplicate problem may need Floyd cycle detection.

20. Final Cheat Sheet

Item	Notes
When to use	Need repeated value detection
Main operations	Check seen, insert, optionally erase old
Complexity	$O(n)$ average
Key code idea	<code>if (seen.count(x)) return true</code>
Edge cases	Empty, $k=0$, all same, no duplicate

Custom hash

1. Overview

Custom hash means defining how non-standard keys like `pair<int,int>`, structs, vectors, or safer integer hashing should be stored in `unordered_map` or `unordered_set`.

It is common in competitive programming for pair keys, coordinate states, graph states, and anti-hash-test protection.

2. Intuition

A hash table needs to convert every key into a number. C++ already knows how to hash `int`, `string`, and some basic types. It does not automatically know how to hash a `pair<int,int>` or your own struct.

So we define a function:

```
key -> hash value
```

Good custom hash should:

- Use all important fields.
- Mix bits well.
- Keep equal keys producing equal hashes.
- Reduce collision chances.

3. When to Use It

Use custom hash when:

- Key is a pair, tuple, vector, or struct.
- You need `unordered_map<pair<int,int>, value>`.
- Problem states are multi-dimensional.
- You store grid coordinates.
- CP problem may contain adversarial input.
- You want faster average lookup than `map<pair<...>>`.

4. When Not to Use It

Avoid custom hash when:

- A simple vector or 2D array works.
- Number of states is tiny.
- You need sorted order; use `map`.
- You are not confident about equality consistency.
- You can compress coordinates into a single integer safely.

5. Core Concepts

Concept	Meaning	Why It Matters
Hash functor	Struct with <code>operator()</code>	Required by <code>unordered_map</code>
Equality	Keys considered same	Hash must be consistent with equality
Bit mixing	Spreading patterns	Reduces collisions
<code>splitmix64</code>	Strong integer mixer	Common CP safe hash
Pair combine	Merge two field hashes	Supports coordinate keys

6. Step-by-Step Algorithm

For custom pair hash:

- 1. Create a struct `PairHash`.
- 2. Define `size_t operator()(const pair<int,int>& p) const`.
- 3. Hash both `p.first` and `p.second`.
- 4. Combine the hashes with bit shifts or a mixer.
- 5. Use it as the third template parameter: `unordered_map<pair<int,int>, int, PairHash>`.

7. Dry Run

Key: `(2, 5)`.

Step	Value	Action
1	2	hash first coordinate
2	5	hash second coordinate
3	both hashes	combine into one hash
4	combined hash	choose bucket
5	key stored	equality still checks exact pair

Final: pair can be used as a hash-map key.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct PairHash {
    size_t operator()(const pair<int, int>& p) const {
        size_t h1 = hash<int>{}(p.first);
        size_t h2 = hash<int>{}(p.second);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
    }
};

struct SafeHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t fixedRandom =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + fixedRandom);
    }
};

int main() {
```

```

unordered_map<pair<int, int>, string, PairHash> cellName;
cellName[{2, 5}] = "blocked";
cellName[{1, 3}] = "free";

cout << cellName[{2, 5}] << '\n';

unordered_map<long long, int, SafeHash> freq;
freq[1000000007LL]++;
cout << freq[1000000007LL] << '\n';

return 0;
}

```

pYTHON IMPLEMENTATION

```

# Python tuples are hashable by default if their elements are hashable.

cell_name = {}
cell_name[(2, 5)] = "blocked"
cell_name[(1, 3)] = "free"

print(cell_name[(2, 5)])

# Custom classes need __hash__ and __eq__ if used as dict/set keys.
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        return isinstance(other, Point) and self.x == other.x and self.y ==
other.y

    def __hash__(self):
        return hash((self.x, self.y))

points = {Point(2, 5): "blocked"}
print(points[Point(2, 5)])

```

10. Code Explanation

- `PairHash` lets `pair<int, int>` be used as a key.
- `hash<int>{}(...)` gets standard hashes for each coordinate.
- The combine formula mixes both fields into one `size_t`.
- `SafeHash` uses `splitmix64` and a random runtime seed to reduce adversarial collision risk.
- Python tuples already support hashing, so `(x, y)` can be used directly as a dictionary key.

11. Complexity Analysis

Operation	Average Time	Worst Time	Space
Custom hash compute	$O(1)$ for pair/int	$O(1)$	$O(1)$
Insert/find in hash map	$O(1)$ average	$O(n)$	$O(n)$ total
Hash vector key	$O(\text{length})$	$O(\text{length})$	$O(1)$ extra

The hash function cost becomes important if the key is large, such as a vector or string.

12. Common Patterns

Pattern	How to Identify	General Approach	Examples
Grid coordinates	Need store (row, col)	Pair hash	Number of Islands variants
State memoization	Multiple parameters define state	Tuple/pair hash	DP state caching
Geometry points	Points as keys	Pair/custom struct hash	Count Points
Anti-hack CP	TLE due to collisions	Safe hash	Codeforces unordered_map hacks

13. Common Mistakes

- Defining hash but forgetting equality for custom structs.
- Hashing only one field of a pair.
- Returning same hash for many keys.
- Using mutable fields in hashed keys and then changing them.
- Thinking custom hash removes all worst-case risk.
- Using `unordered_map<pair<int,int>, int>` without providing a hash in C++17.

14. Edge Cases

- Negative coordinates.
- Very large coordinates.
- Pairs like (a, b) and (b, a) if order matters.
- Struct fields with same values.
- Empty vector keys.
- Mutated keys after insertion.

15. Variations

Variation	What Changes	Use Case	Importance
Pair hash	Hash two fields	Coordinates	Very high
Tuple hash	Hash more fields	Multi-state DP	High
Vector hash	Loop over elements	Sequence state	Medium

Variation	What Changes	Use Case	Importance
Safe integer hash	<code>splitmix64</code>	CP anti-collision	High
Encode to long long	Convert pair to one key	Bounded coordinates	High

16. Related Algorithms/Data Structures

- `map<pair<int,int>, value>`: ordered alternative, $O(\log n)$.
- Coordinate compression: can replace custom hash when coordinates map to small IDs.
- Rolling hash: hashes sequences/strings.
- Universal hashing: theoretical randomized hashing family.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Number of Equivalent Domino Pairs	LeetCode	Pair key count	Easy
Check if Point Exists	Custom/GFG	Pair set	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Max Points on a Line	LeetCode	Pair slope key	Medium/Hard
Number of Islands II	LeetCode	Coordinate states + DSU	Medium
Detect Squares	LeetCode	Point frequency map	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Frog Jump	LeetCode	State pair memo	Hard
Strange Printer II variants	CP	Custom state hashing	Hard
Dynamic Grid Connectivity	Codeforces	Pair hash + DSU	Hard

18. Interview Explanation

Custom hashing is needed when I want to use complex keys, such as pairs or structs, in an unordered map. I define a hash functor that combines the hashes of all fields used for equality. This gives average constant-time lookup while keeping exact equality checks correct.

19. Revision Notes

- Hash must include every equality field.

- Equal keys must produce equal hashes.
- C++17 needs custom hash for pairs.
- Use `splitmix64` in CP when collision attacks are possible.
- Do not mutate keys inside hash containers.

20. Final Cheat Sheet

Item	Notes
When to use	Pair/tuple/struct keys in unordered containers
Main operations	Define hash functor, pass as template arg
Complexity	Hash compute usually $O(1)$
Key code idea	<code>unordered_map<pair<int,int>, int, PairHash></code>
Edge cases	Equality consistency, negative values, collisions

Hashing pairs

1. Overview

Hashing pairs means using a two-value key like `(row, col)`, `(value, index)`, `(x, y)`, or `(numerator, denominator)` in a hash table.

It is a practical custom-hash use case that appears often in grids, geometry, DP, and counting normalized pairs.

2. Intuition

A pair represents a combined identity. For example, cell `(2, 5)` is different from `(5, 2)`.

To store it in a hash table, both parts must influence the final hash. If only one part is used, many different pairs collide.

3. When to Use It

Use pair hashing when:

- You need fast lookup of coordinates.
- State has two parameters.
- You need count of normalized fractions/slopes.
- You store edges `(u, v)`.
- Problem says "grid cell", "coordinate", "pair state", "slope", "interval".

4. When Not to Use It

Avoid it when:

- Coordinates are small enough for a 2D vector.

- Pair can be safely encoded into one integer.
- You need sorted pair order; use `map`.
- Pair is unordered, such as an undirected edge, and you forget to normalize `(min,max)`.
- You need range queries over coordinates.

5. Core Concepts

Concept	Meaning	Example
Ordered pair	<code>(a,b)</code> differs from <code>(b,a)</code>	Directed edge
Unordered pair	<code>(a,b)</code> same as <code>(b,a)</code>	Undirected edge
Normalization	Convert equivalent pairs to same key	slope divided by gcd
Hash combine	Mix both components	Avoid collisions
Encoding	Convert pair to single integer	$x * M + y$

6. Step-by-Step Algorithm

For counting pair frequencies:

1. Decide whether pair order matters.
2. Normalize the pair if needed.
3. Define a pair hash.
4. Use `unordered_map<pair<int,int>, int, PairHash>`.
5. Increment frequency for each pair.
6. Query pair count when needed.

7. Dry Run

Input edges: `(1,2)`, `(2,1)`, `(1,3)` for an undirected graph.

Normalize each pair as `(min(u,v), max(u,v))`.

Edge	Normalized	Count Map
<code>(1,2)</code>	<code>(1,2)</code>	<code>{(1,2):1}</code>
<code>(2,1)</code>	<code>(1,2)</code>	<code>{(1,2):2}</code>
<code>(1,3)</code>	<code>(1,3)</code>	<code>{(1,2):2, (1,3):1}</code>

Final: edge `(1,2)` appears twice.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct PairHash {
```

```

    size_t operator()(const pair<int, int>& p) const {
        size_t h1 = hash<int>{}(p.first);
        size_t h2 = hash<int>{}(p.second);
        return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
    }
};

pair<int, int> normalizeUndirectedEdge(int u, int v) {
    if (u > v) swap(u, v);
    return {u, v};
}

unordered_map<pair<int, int>, int, PairHash>
countUndirectedEdges(const vector<pair<int, int>>& edges) {
    unordered_map<pair<int, int>, int, PairHash> freq;

    for (auto [u, v] : edges) {
        freq[normalizeUndirectedEdge(u, v)]++;
    }

    return freq;
}

int main() {
    vector<pair<int, int>> edges = {{1, 2}, {2, 1}, {1, 3}};
    auto freq = countUndirectedEdges(edges);

    cout << freq[{1, 2}] << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

from collections import Counter

def normalize_undirected_edge(u, v):
    return (u, v) if u <= v else (v, u)

def count_undirected_edges(edges):
    freq = Counter()
    for u, v in edges:
        freq[normalize_undirected_edge(u, v)] += 1
    return freq

edges = [(1, 2), (2, 1), (1, 3)]
print(count_undirected_edges(edges)[(1, 2)])

```

10. Code Explanation

- `PairHash` combines both parts of the pair.

- `normalizeUndirectedEdge` ensures $(1,2)$ and $(2,1)$ become the same key.
- The map stores pair frequencies.
- `freq[{1, 2}]` queries how often the normalized pair appeared.
- Python tuples are hashable, so no custom pair hash is needed.

11. Complexity Analysis

Operation	Time	Space
Normalize pair	$O(1)$	$O(1)$
Hash pair	$O(1)$	$O(1)$
Insert/query	$O(1)$ average	$O(n)$ total
Count all pairs	$O(n)$ average	$O(k)$

k is number of distinct pairs.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Grid visited	(r, c) state	Pair set	BFS/DFS grid
Edge count	(u, v) key	Normalize if undirected	Duplicate edges
Slope grouping	(dy, dx) key	Divide by gcd	Max Points on a Line
DP state	$(index, state)$	Pair memo	Frog Jump

13. Common Mistakes

- Forgetting pair hash in C++ 17.
- Not normalizing undirected pairs.
- Not reducing fractions/slopes by gcd.
- Mishandling signs in slope pairs.
- Encoding $x * M + y$ with too small M , causing collisions.
- Overflow during pair encoding.

14. Edge Cases

- Negative coordinates.
- (a, b) vs (b, a) .
- Self-pair (x, x) .
- Zero denominator in slope representation.
- Very large coordinates.
- Duplicate pairs.

15. Variations

Variation	What Changes	Use Case	Importance
Pair hash	Custom functor	General pairs	Very high
Pair encoding	Convert to integer	Bounded coordinates	High
Normalized pair	Canonical form	Undirected edges/fractions	High
Tuple hashing	More than two values	Larger states	Medium

16. Related Algorithms/Data Structures

- Custom hash: general technique behind pair hashing.
- Coordinate compression: may replace pair hashing for dense IDs.
- DSU: often stores edges or coordinates with pair keys.
- Geometry normalization: slopes and direction vectors are pair keys.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Valid Sudoku	LeetCode	Hash row/col/box pairs	Easy/Medium
Number of Equivalent Domino Pairs	LeetCode	Normalize pair	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Detect Squares	LeetCode	Point pair counts	Medium
Number of Islands	LeetCode	Grid coordinates	Medium
Frog Jump	LeetCode	Pair state memo	Medium/Hard

Hard

Problem	Platform	Main Idea	Difficulty
Max Points on a Line	LeetCode	Normalized slope pairs	Hard
Number of Distinct Islands II	LeetCode	Shape coordinate hashes	Hard
Making A Large Island	LeetCode	Component coordinate logic	Hard

18. Interview Explanation

Hashing pairs lets me store two-dimensional keys like coordinates or state pairs in an unordered map or set. I make sure both components are part of the hash and normalize the pair if equivalent forms should be treated as the same key.

19. Revision Notes

- C++ needs custom pair hash.
- Normalize unordered pairs with `min/max`.
- Normalize slopes with gcd.
- Python tuples work directly.
- Beware encoding overflow.

20. Final Cheat Sheet

Item	Notes
When to use	Coordinates, edges, two-parameter states
Main operations	Normalize, hash, insert/query
Complexity	Average $O(1)$ per operation
Key code idea	<code>unordered_map<pair<int,int>, int, PairHash></code>
Edge cases	Order, signs, zero, overflow

Coordinate compression

1. Overview

Coordinate compression maps large or sparse values to small consecutive indices while preserving relative order.

Example:

```
values:  [1000000000, -5, 42]
sorted:  [-5, 42, 1000000000]
mapped:  -5 -> 0, 42 -> 1, 1000000000 -> 2
```

It is used when values are too large for direct indexing but only their order or identity matters.

2. Intuition

If only relative order matters, the exact values are unnecessary.

Analogy: In a race, instead of storing exact finish times, we store ranks. Rank `1, 2, 3` is easier to index than huge times.

Compression works because comparisons are preserved:

```
a < b  =>  compressed[a] < compressed[b]
```

3. When to Use It

Use coordinate compression when:

- Values are huge but number of distinct values is small.
- You need use Fenwick tree or segment tree on values.
- Problem asks inversions, order statistics, range counts.
- Coordinates are up to $1e9$ but n is only $2e5$.
- You need convert arbitrary labels to IDs.

Trigger phrases:

- "large coordinates"
- "compress values"
- "range queries over values"
- "coordinate up to 10^9 "
- "count smaller/greater"

4. When Not to Use It

Avoid it when:

- Actual distances matter, not just order.
- Values are already small enough.
- You need preserve gaps between coordinates.
- Online new values appear and cannot be preprocessed; use ordered maps or dynamic segment tree.
- Compression would break arithmetic meaning, such as difference $x - y$.

5. Core Concepts

Concept	Meaning	Why It Matters
Unique sorted values	Sorted list without duplicates	Defines compressed order
Rank/index	New small ID	Enables array/Fenwick indexing
Lower bound	Finds rank of value	$O(\log n)$ mapping
Hash map mapping	Value to compressed ID	$O(1)$ average after build
Order preservation	Relative comparison unchanged	Critical for range structures

6. Step-by-Step Algorithm

1. Copy all values that may appear.
2. Sort the copy.
3. Remove duplicates.
4. For each original value, find its index in the sorted unique list.
5. Use this index as the compressed coordinate.
6. If using Fenwick tree, usually add 1 to make it 1-indexed.

7. Dry Run

Input: [50, 10, 50, 100, -20].

Step	Result
Copy	[50, 10, 50, 100, -20]
Sort	[-20, 10, 50, 50, 100]
Unique	[-20, 10, 50, 100]
Mapping	-20->0, 10->1, 50->2, 100->3
Compressed array	[2, 1, 2, 3, 0]

Final compressed values preserve ordering.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> compressValues(const vector<long long>& values) {
    vector<long long> sortedValues = values;
    sort(sortedValues.begin(), sortedValues.end());
    sortedValues.erase(unique(sortedValues.begin(), sortedValues.end()),
sortedValues.end());

    vector<int> compressed;
    compressed.reserve(values.size());

    for (long long x : values) {
        int id = lower_bound(sortedValues.begin(), sortedValues.end(), x) -
sortedValues.begin();
        compressed.push_back(id);
    }

    return compressed;
}

unordered_map<long long, int> buildCompressionMap(const vector<long long>& values)
{
    vector<long long> sortedValues = values;
    sort(sortedValues.begin(), sortedValues.end());
    sortedValues.erase(unique(sortedValues.begin(), sortedValues.end()),
sortedValues.end());

    unordered_map<long long, int> id;
    id.reserve(sortedValues.size() * 2);

    for (int i = 0; i < (int)sortedValues.size(); i++) {
        id[sortedValues[i]] = i;
    }
}
```

```

    }

    return id;
}

int main() {
    vector<long long> values = {50, 10, 50, 100, -20};
    vector<int> compressed = compressValues(values);

    for (int id : compressed) {
        cout << id << ' ';
    }
    cout << '\n';

    return 0;
}

```

pYTHON IMPLEMENTATION

```

def compress_values(values):
    sorted_values = sorted(set(values))
    id_by_value = {value: i for i, value in enumerate(sorted_values)}
    return [id_by_value[x] for x in values]

print(compress_values([50, 10, 50, 100, -20]))

```

10. Code Explanation

- Copying protects original input order.
- Sorting arranges values by rank.
- `unique` removes duplicates so equal values get the same ID.
- `lower_bound` finds the rank of each original value.
- `buildCompressionMap` is faster for repeated mapping after preprocessing.
- Python uses `sorted(set(values))` to get unique sorted coordinates.

11. Complexity Analysis

Step	Time	Space
Copy values	$O(n)$	$O(n)$
Sort	$O(n \log n)$	depends on sort
Unique	$O(n)$	$O(1)$ extra
Map with <code>lower_bound</code>	$O(n \log n)$	$O(n)$ output
Map with hash map	$O(n)$ average after build	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Fenwick over values	Values up to $1e9$	Compress to $1..m$	Count Inversions
Segment tree over coordinates	Sparse coordinates	Compress endpoints	Range union
Label to ID	Strings/large IDs	Map labels to ints	Graph with city names
Count smaller	Need rank	Compress + BIT	Count Smaller After Self

13. Common Mistakes

- Compressing when actual distance between coordinates matters.
- Forgetting $+1$ for 1-indexed Fenwick tree.
- Missing coordinates that appear only in queries.
- Not removing duplicates.
- Using compressed value in arithmetic difference.
- Not including interval endpoints carefully for range problems.

14. Edge Cases

- Empty values.
- All equal values.
- Negative coordinates.
- Very large values.
- Duplicate values.
- Query values not in original list.
- Intervals requiring $r+1$ endpoints.

15. Variations

Variation	What Changes	Use Case	Importance
0-indexed compression	IDs $0..m-1$	Arrays/vectors	High
1-indexed compression	IDs $1..m$	Fenwick tree	Very high
Label compression	Strings to IDs	Graphs/maps	High
2D compression	Compress x and y separately	Geometry/grid	Medium
Dynamic compression	Ordered map as values arrive	Online problems	Medium/Hard

16. Related Algorithms/Data Structures

- Fenwick tree: often needs compressed indices.
- Segment tree: compression reduces coordinate universe.
- Hash map: maps original values to compressed IDs.
- Sorting: required to preserve order.
- Ordered set: alternative for online rank queries, but harder.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Rank Transform of an Array	LeetCode	Coordinate compression	Easy
Coordinate Compression	AtCoder practice	Sort unique mapping	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Count Inversions	GFG/CSES	Compression + Fenwick	Medium
Nested Ranges Count	CSES	Compression + sorting	Medium
Range Sum Query variants	Codeforces	Compress sparse coords	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Count of Smaller Numbers After Self	LeetCode	Compression + Fenwick	Hard
Rectangle Area II	LeetCode	Coordinate compression + sweep	Hard
Posters	SPOJ	Interval compression	Hard

18. Interview Explanation

Coordinate compression replaces large sparse values with small ranks while preserving order. I sort unique values, assign each value an index, and use that index in arrays, Fenwick trees, or segment trees. I only use it when exact gaps between values are not important.

19. Revision Notes

- Sort + unique.
- Rank is index in sorted unique array.
- Use `+1` for Fenwick.
- Do not use compressed IDs for distance arithmetic.
- Include all query/update coordinates before compressing.

20. Final Cheat Sheet

Item	Notes
When to use	Large sparse values needing indexing
Main operations	Sort, unique, lower_bound/map
Complexity	$O(n \log n)$ preprocessing
Key code idea	<code>id = lower_bound(coords, x)</code>

Item	Notes
Edge cases	Duplicates, missing query coords, actual gaps

Rolling hash

1. Overview

Rolling hash converts a string or sequence into numeric hash values so substring hashes can be computed quickly. It is widely used for string matching, duplicate substring detection, and comparing substrings.

The common polynomial hash is:

$$\text{hash}(s) = s[0] \cdot p^0 + s[1] \cdot p^1 + \dots + s[n-1] \cdot p^{(n-1)} \bmod M$$

2. Intuition

A rolling hash gives each string a compact fingerprint.

For substring queries:

- Precompute prefix hashes.
- Precompute powers of a base.
- Use subtraction to get substring hash in $O(1)$.

Analogy: Instead of comparing every character of two long strings, compare their fingerprints first.

3. When to Use It

Use rolling hash when:

- Need compare many substrings quickly.
- Need find duplicate substrings.
- Need pattern matching.
- Need binary search on substring length.
- Problem says "substring equality", "repeated substring", "longest duplicate substring", "string matching".

4. When Not to Use It

Avoid it when:

- Exact deterministic matching is required and collisions are unacceptable; use KMP/Z/suffix array.
- Only one pattern match is needed; KMP may be safer.
- String length is tiny.
- You cannot tolerate modular collision risk.
- You need lexicographic ordering; suffix array/tree/trie may be better.

5. Core Concepts

Concept	Meaning	Why It Matters
Base	Multiplier for positions	Spreads characters
Modulus	Keeps hash bounded	Prevents overflow
Prefix hash	Hash of prefix	Enables substring hash
Power array	Powers of base	Aligns substring positions
Collision	Different strings same hash	Main risk

6. Step-by-Step Algorithm

For substring hash:

- 1. Choose base `p` and modulus `M`.
- 2. Precompute `power[i] = p^i mod M`.
- 3. Build prefix hash where `prefix[i+1] = prefix[i]*p + value(s[i])`.
- 4. To hash substring `[l, r]`, compute: `prefix[r+1] - prefix[l] * power[r-l+1]`.
- 5. Normalize by adding modulus before `%`.
- 6. Compare substring hashes.

7. Dry Run

String: `"abcd"`, base `31`, mod large.

Use forward formula:

```
prefix[0] = 0
prefix[i+1] = prefix[i] * 31 + value(s[i])
```

Let `a=1, b=2, c=3, d=4`.

i	char	prefix calculation	prefix
0	a	<code>0*31 + 1</code>	1
1	b	<code>1*31 + 2</code>	33
2	c	<code>33*31 + 3</code>	1026
3	d	<code>1026*31 + 4</code>	31810

Hash of `"bc"` from `[1,2]`:

```
prefix[3] - prefix[1] * 31^2
= 1026 - 1 * 961
```

```
= 65
```

This equals $b \cdot 31 + c = 2 \cdot 31 + 3 = 65$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class RollingHash {
private:
    static const long long MOD = 1000000007LL;
    static const long long BASE = 911382323LL;
    vector<long long> prefix;
    vector<long long> power;

public:
    RollingHash(const string& s) {
        int n = s.size();
        prefix.assign(n + 1, 0);
        power.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            power[i + 1] = (power[i] * BASE) % MOD;
            prefix[i + 1] = (prefix[i] * BASE + s[i]) % MOD;
        }

        long long getHash(int l, int r) const {
            long long result = (prefix[r + 1] - (prefix[l] * power[r - l + 1]) % MOD +
MOD) % MOD;
            return result;
        }
    };

    int main() {
        string s = "abcdabc";
        RollingHash rh(s);

        cout << (rh.getHash(0, 2) == rh.getHash(4, 6)) << '\n'; // "abc" == "abc"
        cout << (rh.getHash(1, 2) == rh.getHash(2, 3)) << '\n'; // "bc" != "cd"

        return 0;
    }
}
```

pYTHON IMPLEMENTATION

```
class RollingHash:
    MOD = 1_000_000_007
```

```

BASE = 911382323

def __init__(self, s):
    n = len(s)
    self.prefix = [0] * (n + 1)
    self.power = [1] * (n + 1)

    for i, ch in enumerate(s):
        self.power[i + 1] = (self.power[i] * self.BASE) % self.MOD
        self.prefix[i + 1] = (self.prefix[i] * self.BASE + ord(ch)) % self.MOD

    def get_hash(self, l, r):
        length = r - l + 1
        return (self.prefix[r + 1] - self.prefix[l] * self.power[length]) %
self.MOD

rh = RollingHash("abcdabc")
print(rh.get_hash(0, 2) == rh.get_hash(4, 6))

```

10. Code Explanation

- `prefix[i]` stores the hash of `s[0..i-1]`.
- `power[i]` stores `BASEi mod MOD`.
- `getHash(l,r)` removes the contribution of characters before `l`.
- Adding `MOD` before `% MOD` prevents negative hash values in C++.
- Comparing two hashes gives a probabilistic equality check.

11. Complexity Analysis

Operation	Time	Space
Precompute prefix and powers	$O(n)$	$O(n)$
Substring hash query	$O(1)$	$O(1)$
Compare two substrings	$O(1)$	$O(1)$
Pattern scan with hashes	$O(n + m)$	$O(n)$

Collision risk exists even with good parameters.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Substring equality	Many equality queries	Prefix rolling hash	Palindrome/string queries
Pattern matching	Find occurrences	Compare pattern hash	Rabin-Karp
Duplicate substring	Repeated substrings	Binary search length + hash set	Longest Duplicate Substring

Pattern	How to Identify	Approach	Examples
Distinct substrings	Count unique hashes	Hash all lengths	CP string tasks

13. Common Mistakes

- Forgetting to normalize negative modulo.
- Using one small modulus and getting collisions.
- Mixing 0-indexed and 1-indexed prefix positions.
- Comparing hashes of differently aligned formulas incorrectly.
- Using `char` directly when signedness matters; cast or use `ord`.
- Choosing bad base like `1`.

14. Edge Cases

- Empty string.
- One-character substring.
- Whole string query.
- Repeated same character.
- Very long strings.
- Collision cases.
- Non-lowercase characters.

15. Variations

Variation	What Changes	Use Case	Importance
Rabin-Karp	Rolling pattern search	String matching	High
Double rolling hash	Two moduli	Lower collision risk	Very high
64-bit hash	Natural unsigned overflow	Fast CP hashing	Medium
Reverse hash	Hash reversed string	Palindrome queries	High
2D rolling hash	Hash matrices	Grid pattern matching	Medium

16. Related Algorithms/Data Structures

- KMP: deterministic pattern matching without collision.
- Z Algorithm: deterministic prefix matching.
- Suffix array: sorted suffixes and substring queries.
- Trie: prefix storage and search.
- Double hashing: safer rolling hash.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Repeated Substring Pattern	LeetCode	String matching/hash alternative	Easy
Implement strStr	LeetCode	Rabin-Karp/KMP	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Repeated DNA Sequences	LeetCode	Hash fixed-length substrings	Medium
Distinct Echo Substrings	LeetCode	Substring hash compare	Medium
Find Duplicate Subtrees	LeetCode	Hash serialized structures	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Binary search + rolling hash	Hard
Palindrome Queries	Codeforces	Forward/reverse hashes	Hard
String Matching With Wildcards	CP	Hash segments	Hard

18. Interview Explanation

Rolling hash precomputes prefix hashes and powers so I can get any substring hash in constant time. It is useful for fast substring comparison and Rabin-Karp-style matching. Since hashes can collide, I mention collision risk and use double hashing if correctness needs stronger practical safety.

19. Revision Notes

- Formula: $\text{hash}(l, r) = \text{pref}[r+1] - \text{pref}[l] * \text{base}^{\text{len}}$.
- Add MOD before %.
- Precompute powers.
- Hash comparison is probabilistic.
- Use double hashing for safer CP solutions.

20. Final Cheat Sheet

Item	Notes
When to use	Many substring comparisons/searches
Main operations	Precompute prefix/power, query substring hash
Complexity	Build $O(n)$, query $O(1)$
Key code idea	$\text{pref}[r+1] - \text{pref}[l] * \text{pow}[\text{len}]$
Edge cases	Negative modulo, collisions, index boundaries

Double hashing

1. Overview

Double hashing uses two independent hash values for the same key or substring. In string hashing, it usually means using two different moduli and/or bases.

It greatly reduces the chance that two different strings produce the same hash pair.

2. Intuition

One fingerprint can accidentally match. Two independent fingerprints matching is much less likely.

If hash1 collides with probability around $1/M1$ and hash2 collides with probability around $1/M2$, both colliding is roughly $1/(M1*M2)$ under reasonable independence.

3. When to Use It

Use double hashing when:

- Rolling hash collision risk matters.
- Need compare many substrings.
- Binary searching duplicate substrings.
- CP judge is strict and adversarial tests may exist.
- You need practical reliability without suffix arrays.

4. When Not to Use It

Avoid it when:

- Deterministic correctness is required; use KMP/Z/suffix array/tree.
- Problem is simple and one hash is enough as a filter with verification.
- Performance/memory is extremely tight.
- You are already using exact string comparison after hash match.

5. Core Concepts

Concept	Meaning	Why It Matters
Two moduli	Compute hash under two primes	Reduces collision
Hash pair	Store $(h1, h2)$	Both must match
Same substring formula	Applied twice	Implementation symmetry
Collision probability	Chance of false equality	Much smaller
Pair hash	Needed to store hash pairs in C++ unordered set	CP implementation

6. Step-by-Step Algorithm

1. Choose two large moduli.
2. Choose a base.
3. Precompute prefix hashes under both moduli.
4. Precompute powers under both moduli.
5. For substring $[l, r]$, compute both hash values.
6. Compare/store the pair $(hash1, hash2)$.

7. Dry Run

String: "abcabc".

Compare substrings $[0, 2]$ and $[3, 5]$, both "abc".

Query	Substring	Hash 1	Hash 2	Result
$[0, 2]$	abc	H1a	H2a	pair A
$[3, 5]$	abc	H1a	H2a	pair A

Both components match, so substrings are considered equal.

If only one component matched, we would treat them as different.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DoubleRollingHash {
private:
    static const long long MOD1 = 1000000007LL;
    static const long long MOD2 = 1000000009LL;
    static const long long BASE = 911382323LL;

    vector<long long> pref1, pref2, pow1, pow2;

public:
    DoubleRollingHash(const string& s) {
        int n = s.size();
        pref1.assign(n + 1, 0);
        pref2.assign(n + 1, 0);
        pow1.assign(n + 1, 1);
        pow2.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            pow1[i + 1] = (pow1[i] * BASE) % MOD1;
            pow2[i + 1] = (pow2[i] * BASE) % MOD2;

            pref1[i + 1] = (pref1[i] * BASE + s[i]) % MOD1;
            pref2[i + 1] = (pref2[i] * BASE + s[i]) % MOD2;
        }
    }
}
```

```

pair<long long, long long> getHash(int l, int r) const {
    int len = r - l + 1;

    long long h1 = (pref1[r + 1] - pref1[l] * pow1[len] % MOD1 + MOD1) % MOD1;
    long long h2 = (pref2[r + 1] - pref2[l] * pow2[len] % MOD2 + MOD2) % MOD2;

    return {h1, h2};
}

};

int main() {
    string s = "abcabc";
    DoubleRollingHash hash(s);

    cout << boolalpha << (hash.getHash(0, 2) == hash.getHash(3, 5)) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class DoubleRollingHash:
    MOD1 = 1_000_000_007
    MOD2 = 1_000_000_009
    BASE = 911382323

    def __init__(self, s):
        n = len(s)
        self.pref1 = [0] * (n + 1)
        self.pref2 = [0] * (n + 1)
        self.pow1 = [1] * (n + 1)
        self.pow2 = [1] * (n + 1)

        for i, ch in enumerate(s):
            value = ord(ch)
            self.pow1[i + 1] = self.pow1[i] * self.BASE % self.MOD1
            self.pow2[i + 1] = self.pow2[i] * self.BASE % self.MOD2
            self.pref1[i + 1] = (self.pref1[i] * self.BASE + value) % self.MOD1
            self.pref2[i + 1] = (self.pref2[i] * self.BASE + value) % self.MOD2

    def get_hash(self, l, r):
        length = r - l + 1
        h1 = (self.pref1[r + 1] - self.pref1[l] * self.pow1[length]) % self.MOD1
        h2 = (self.pref2[r + 1] - self.pref2[l] * self.pow2[length]) % self.MOD2
        return (h1, h2)

h = DoubleRollingHash("abcabc")
print(h.get_hash(0, 2) == h.get_hash(3, 5))

```

10. Code Explanation

- Two prefix arrays are maintained, one per modulus.
- Two power arrays are maintained for the same reason.
- `getHash` computes the substring hash under both moduli.
- The returned pair is compared as a combined fingerprint.
- Python tuples naturally store the hash pair.

11. Complexity Analysis

Operation	Time	Space
Precompute	$O(n)$	$O(n)$
Substring hash	$O(1)$	$O(1)$
Compare substrings	$O(1)$	$O(1)$
Store substring hashes	$O(n)$ average	$O(n)$

Double hashing roughly doubles constant factors compared to single hashing.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Safer substring equality	Many comparisons	Pair hash	CP string queries
Duplicate substring	Collision-sensitive	Binary search + double hash	Longest Duplicate Substring
Palindrome query	Forward/reverse compare	Double hash both directions	Palindrome Substring Queries
2D string/grid hash	More collision risk	Double hash rows/columns	Grid matching

13. Common Mistakes

- Accidentally using same modulus twice.
- Forgetting to normalize negative values for both hashes.
- Comparing only one component.
- Using pair in `unordered_set` without pair hash in C++.
- Choosing base equal to or larger than modulus in bad ways.
- Assuming collision probability is exactly zero.

14. Edge Cases

- Empty string.
- Same repeated characters.
- Full-length substring.
- Adjacent substrings.

- Very long strings.
- Hash pair storage collisions if pair hash is poor.

15. Variations

Variation	What Changes	Use Case	Importance
Two moduli, one base	Common double hash	String CP	Very high
Two bases, one modulus	Alternative independence	Less common	Medium
64-bit + mod hash	Fast plus safer	CP optimization	Medium
Triple hashing	Three components	Extreme collision caution	Low/Medium

16. Related Algorithms/Data Structures

- Rolling hash: single-hash base technique.
- Pair hashing: needed to store hash pairs.
- KMP/Z: deterministic string matching.
- Suffix array: deterministic duplicate substring and ordering.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Implement strStr	LeetCode	Rabin-Karp option	Easy
Repeated Substring Pattern	LeetCode	Hash/string trick	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Repeated DNA Sequences	LeetCode	Store substring hashes	Medium
Distinct Echo Substrings	LeetCode	Compare adjacent hashes	Medium
Palindrome Queries	Codeforces	Forward/reverse hashes	Medium/Hard

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Binary search + double hash	Hard
Pattern Matching with Mismatches	CP	Hash segments	Hard
Distinct Substrings Large Constraints	CSES/CP	Hash or suffix array	Hard

18. Interview Explanation

Double hashing computes two independent hash values for the same substring. I compare the pair instead of a single number, which makes accidental collisions much less likely. It is still probabilistic, but in competitive programming it is often reliable enough when implemented carefully.

19. Revision Notes

- Use two large prime moduli.
- Return pair `(h1,h2)`.
- Normalize both hashes.
- Collision chance becomes tiny, not zero.
- Need pair hash for unordered sets in C++.

20. Final Cheat Sheet

Item	Notes
When to use	Safer substring hashing
Main operations	Compute two prefix hashes
Complexity	Build $O(n)$, query $O(1)$
Key code idea	<code>return {hash1(l,r), hash2(l,r)}</code>
Edge cases	Negative modulo, pair storage, collision assumptions

Universal hashing basics

1. Overview

Universal hashing is a randomized hashing idea where the hash function is chosen from a family of functions. The goal is to reduce the chance that an adversary can force many collisions.

It is more theoretical than everyday `unordered_map`, but the idea explains why randomized hashes like `splitmix64` are useful in competitive programming.

2. Intuition

If a fixed hash function is known, someone can craft keys that collide. Universal hashing chooses a function randomly from a large family, making it hard to predict collisions.

Analogy: If everyone knows which locker rule is used, they can choose items that go to the same locker. If the locker rule changes randomly, planned collisions become unlikely.

3. When to Use It

Use the idea when:

- You need protection against adversarial hash collisions.
- You are solving Codeforces/CP problems where `unordered_map` can be hacked.
- You design a hash table.

- You need understand expected performance guarantees.
- You use randomized hashing for robust lookup.

4. When Not to Use It

Do not overuse it when:

- Standard containers are enough and input is not adversarial.
- You need cryptographic security; universal hashing is not automatically cryptographic.
- You need deterministic reproducibility.
- Simpler arrays/vectors solve the problem.
- You need sorted order.

5. Core Concepts

Concept	Meaning	Why It Matters
Hash family	Set of possible hash functions	Choose one randomly
Random seed	Runtime randomness	Makes collisions hard to predict
Collision probability	Chance two keys collide	Universal families bound it
Pairwise independence	Two keys behave independently enough	Gives expected performance
Non-cryptographic	Not meant for security	Do not use for passwords

Classic family:

$$h(x) = ((a*x + b) \bmod p) \bmod m$$

where p is prime, m is table size, and a, b are random.

6. Step-by-Step Algorithm

For a simple universal hash over integers:

1. Choose a prime p larger than possible key values.
2. Choose table size m .
3. Randomly choose a from $[1, p-1]$.
4. Randomly choose b from $[0, p-1]$.
5. Compute $h(x) = ((a*x + b) \% p) \% m$.
6. Use $h(x)$ as the bucket index.

7. Dry Run

Let:

$p = 101, m = 10, a = 7, b = 3$
 $h(x) = ((7x + 3) \bmod 101) \bmod 10$

x	7x+3	mod 101	bucket
5	38	38	8
12	87	87	7
20	143	42	2

Final: keys are distributed according to the chosen random function.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class UniversalIntHash {
private:
    long long a, b;
    long long prime;
    long long bucketCount;

public:
    UniversalIntHash(long long bucketCount, long long prime = 1000000007LL)
        : prime(prime), bucketCount(bucketCount) {
        mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
        uniform_int_distribution<long long> distA(1, prime - 1);
        uniform_int_distribution<long long> distB(0, prime - 1);
        a = distA(rng);
        b = distB(rng);
    }

    long long operator()(long long x) const {
        x %= prime;
        if (x < 0) x += prime;
        return ((a * x + b) % prime) % bucketCount;
    }
};

struct SafeHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {

```

```

        static const uint64_t seed =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};

int main() {
    UniversalIntHash bucketHash(10);
    cout << bucketHash(5) << '\n';
    cout << bucketHash(12) << '\n';

    unordered_map<long long, int, SafeHash> freq;
    freq[1234567890123LL]++;
    cout << freq[1234567890123LL] << '\n';

    return 0;
}

```

pYTHON IMPLEMENTATION

```

import random

class UniversalIntHash:
    def __init__(self, bucket_count, prime=1_000_000_007):
        self.prime = prime
        self.bucket_count = bucket_count
        self.a = random.randint(1, prime - 1)
        self.b = random.randint(0, prime - 1)

    def __call__(self, x):
        x %= self.prime
        return ((self.a * x + self.b) % self.prime) % self.bucket_count

h = UniversalIntHash(10)
print(h(5))
print(h(12))

```

10. Code Explanation

- `a` and `b` define the randomly selected hash function.
- `prime` keeps arithmetic in a finite field-like range.
- `bucketCount` maps the hash to table buckets.
- Negative values are normalized before hashing.
- `SafeHash` is the CP-friendly practical version used with `unordered_map`.

11. Complexity Analysis

Operation	Time	Space
Choose hash function	$O(1)$	$O(1)$
Hash one integer	$O(1)$	$O(1)$
Hash table insert/find expected	$O(1)$	$O(n)$ total
Worst-case with unlucky collisions Can degrade Depends on table		

Universal hashing improves expected behavior but does not make collisions impossible.

12. Common Patterns

Pattern	How to Identify	Approach	Examples
Anti-collision map	<code>unordered_map</code> TLE on CP	Randomized safe hash	Codeforces maps
Hash table design	Implement hash table	Choose random function	DS theory
Randomized algorithms	Avoid adversarial input	Random seed/mixer	CP robustness
Large integer keys	Need spread bits	<code>splitmix64</code>	Frequency maps

13. Common Mistakes

- Treating universal hashing as cryptographic hashing.
- Choosing $a = 0$, which destroys distribution.
- Using non-prime modulus in formulas that assume prime.
- Forgetting negative key normalization.
- Reusing predictable fixed bad hashes in adversarial settings.
- Thinking randomized hashing guarantees no collisions.

14. Edge Cases

- Negative keys.
- Key larger than prime.
- Bucket count `1`.
- Very many keys compared to buckets.
- Reproducibility needed for debugging.
- Overflow in $a*x+b$; use 128-bit if constraints exceed safe range.

15. Variations

Variation	What Changes	Use Case	Importance
Multiply-add-mod-prime	Classic universal family	Theory/basic hash table	Medium
<code>splitmix64</code> safe hash	Practical bit mixer	CP <code>unordered_map</code>	Very high
Tabulation hashing	Random tables for bytes	Advanced hashing	Low/Medium
Cryptographic hashing	Security-focused	Password/signature contexts	Not CP

16. Related Algorithms/Data Structures

- Custom hash: practical way to use safer hash functions.
- `unordered_map`: benefits from good hash distribution.
- Rolling hash: hashes strings but for different purpose.
- Bloom filter: uses multiple hash functions for probabilistic membership.
- Randomized algorithms: universal hashing is a core example.

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Frequency Count	GFG	Hash map basics	Easy
Contains Duplicate	LeetCode	Hash set lookup	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Subarray Sum Equals K	LeetCode	Hash map under large input	Medium
Detect Squares	LeetCode	Pair/count hashing	Medium
Codeforces unordered_map hacks	Codeforces blogs/tasks	Safe hash	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Longest Duplicate Substring	LeetCode	Collision-aware hashing	Hard
Dynamic State Counting	Codeforces	Safe custom hash	Hard
Large Coordinate Pair Counting	CP	Pair hash + randomized mixer	Hard

18. Interview Explanation

Universal hashing means choosing a hash function randomly from a family so that no fixed adversarial input can reliably cause many collisions. In practice, for CP, I use randomized safe hashes such as `splitmix64` to protect `unordered_map` from bad collision cases.

19. Revision Notes

- Classic formula: $h(x) = ((a * x + b) \% p) \% m$.
- Choose `a` non-zero.
- Randomization reduces adversarial collision risk.
- Not cryptographic.
- `splitmix64` is practical for CP.

20. Final Cheat Sheet

Item	Notes
When to use	Robust hashing against adversarial inputs
Main operations	Randomly choose hash parameters, hash keys
Complexity	Expected $O(1)$ hash table operations
Key code idea	Random seed + strong mixer
Edge cases	Negative keys, overflow, reproducibility, collisions